

Aircade Development Roadmap

Executive Summary

Aircade is a retro-style 2D pixel-art survival adventure designed for offline play. In the game, the player survives a plane crash and must scavenge wreckage, manage health/hunger/thirst/stamina, craft tools, and overcome environmental hazards to escape. The game features a clear narrative (e.g. find water, build a shelter, repair a radio, activate a rescue beacon) rather than endless survival, ensuring each objective advances the story. As an **offline-first** title, **all single-player content requires no internet** (ideal for flights, trains, remote areas) [20†L124-L133] . Crucially, Aircade supports **local Bluetooth P2P multiplayer**: nearby devices connect directly (Host-Client model) without Wi-Fi, letting friends cooperatively or competitively survive together. This dual focus – a polished offline solo experience plus seamless Bluetooth-based local multiplayer – makes Aircade uniquely suited for travel and offline scenarios while keeping development fully free/open-source.

Technology Comparison

Game Engines Comparison

Engine	Primary Language(s)	Platforms	License	Initial Release
Phaser 3	JavaScript / TypeScript [1†L93-L100]	Web (HTML5/Browser); can package to Android/iOS (via Cordova/Electron)	MIT [2†L7-L10]	2013
Godot 4	GScript (Python-like), C#, C++ [1†L100-L107]	Desktop (Windows/Linux/macOS), Mobile (Android/iOS), WebGL, consoles	MIT [2†L7-L10]	2014 (public 2018)
GDevelop 5	JS/TypeScript/C++ (engine) [22†L68-L74]	Desktop (Windows/macOS/Linux), Web (HTML5), Mobile (Android/iOS)	MIT [7†L330-L338]	2013

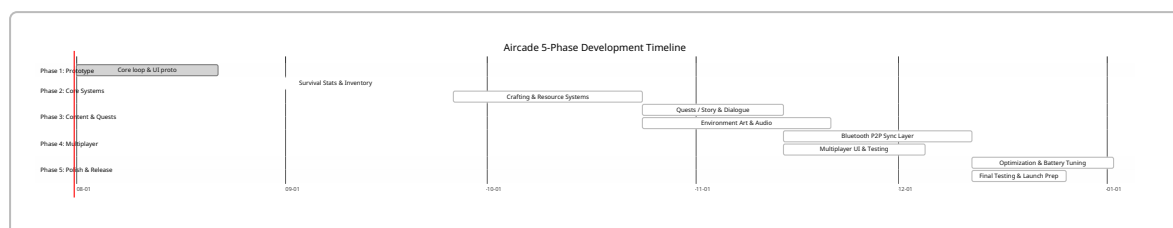
- **Languages:** Phaser games are coded in JS/TS [1†L93-L100] , Godot uses its built-in GScript or C#/C++, and GDevelop's engine core uses JS/TS/C++ [22†L68-L74] .
- **Platforms:** All engines can target mobile and web. Godot and GDevelop have built-in export tools to Android APK, iOS (via Xcode), HTML5/WebGL, plus desktop builds. Phaser is browser-native (zero install) and can be wrapped for Android/iOS via Cordova or Electron.
- **License:** Each engine is fully open-source (MIT licence) [2†L7-L10] [7†L330-L338] , with no royalties or fees.
- **Editor:** Godot offers a complete IDE (scene editor, debugger, profiler); Phaser is code-first (you use external code editors); GDevelop is an event-based visual editor.
- **Maturity:** All are mature, widely-used engines (Godot >10 years, Phaser since 2013, GDevelop since ~2013), with active communities and extensive documentation.

Bluetooth P2P Libraries/Plugins

Library / API	Platforms	License	Maturity / Notes
Web Bluetooth API	Modern browsers (Chrome/Edge on desktop & Android)	Web Standard (open)	Limited support (no Safari/iOS). Designed for BLE device access, <i>not</i> for browser-to-browser multiplayer 【13†L25-L28】 .
Cordova Bluetooth LE	Android, iOS (via Cordova/ Capacitor)	Apache-2.0 【35†L416-L424】	Mature, widely-used plugin for BLE. Works with any HTML5 engine (Phaser/GDevelop).
React Native BLE PLX	Android, iOS	Apache-2.0 【39†L259-L262】	Well-maintained React Native library (not directly used here, but representative of mature BLE libs). Supports central/peripheral roles.
Android Bluetooth API	Android devices (Java/Kotlin)	Apache-2.0 (built-in)	Native Android SDK (Classic and BLE); full support for P2P sockets.
MultipeerConnectivity	iOS (Apple devices)	Proprietary (Apple)	Built-in iOS framework for peer-to-peer (BLE/Wi-Fi); closed but high-level.

- **Web Bluetooth:** Built into Chrome/Edge, but currently only allows connecting to BLE peripherals (e.g. IoT devices), *not* to other phones running the game 【13†L25-L28】 . In practice, true P2P between browsers is not supported, so on-the-fly multiplayer must use native Bluetooth or a mesh network.
- **Cordova Bluetooth LE:** A proven plugin (Apache-2.0) that lets a JavaScript app scan/connect over BLE 【35†L416-L424】 . Use this for Android/iOS builds with Phaser or GDevelop.
- **Native APIs:** Alternatively, for Godot or native layers, one could implement Bluetooth via platform SDKs. We plan a simplified host-client model (one device as Host/server, others as Clients) and implement client-side prediction and server reconciliation to sync state.

Development Timeline



Phase 1: Prototype & Core Loop

- **Goal:** Establish a working single-player prototype demonstrating basic movement, environment, and the survival stats. Verify the core gameplay loop (explore, gather, survive) is fun on its own.
- **Duration:** ~3 weeks.

- **Deliverables:** Prototype build with player movement (8-directional), basic collision, UI showing Health/Hunger/Thirst bars, and a small test level (e.g. a section of island or forest) with a few pick-up items (wood, food, water).
- **Team Tasks:**
- **Dhruv (OpenCode/Nemtron/NVIDIA):** Use the **OpenCode** CLI with an NVIDIA AI model to scaffold the project (create initial files, set up Phaser or Godot scene). Prompt **Nemtron** to generate boilerplate for player movement and camera control code. Refine collision detection and input handling. Use NVIDIA AI (e.g. an image-generation model) to create placeholder 8-bit sprites (or generate them via code and convert to pixel art). Implement basic UI HUD (health bar, icons) by asking Nemtron for code examples. Write initial automated tests for player movement using OpenCode's test generation capabilities.
- **Milind (Antigravity):** Set up the development environment on mobile/hardware. Configure a test device (Android phone) and ensure remote debugging tools work. With **Antigravity AI IDE**, generate and refine code for touch controls or virtual joystick. Help integrate Dhruv's code into an APK build (using Cordova/Android Studio). Use Antigravity to write an automated build script that exports the HTML5 game as an Android APK. Prototype saving player stats in local storage or file (using plugin or engine API).
- **Vinayak (Antigravity):** Use **Antigravity AI** to design the first UI mockups (inventory screen, status bars, simple menus) and produce matching pixel-art icons (water drop, apple, etc.). Implement these assets in the engine: add the HUD overlay, text labels, and placeholder inventory slot. Set up the initial scene (camera, terrain tiles) using engine tools or GDevelop events. Write placeholder quest text (e.g. "Find water!") and implement a simple event to display it. Document the prototype's scope and usage for the next phase.
- **Tools/Libraries/Languages:** Phaser 3 (JavaScript/TypeScript) or Godot 4 (GDScript/C#) as chosen engine; Git/GitHub; VS Code; OpenCode CLI (with OpenAI/GPT or Claude models); Nemtron agent; NVIDIA model (for assets/code); Antigravity IDE; HTML5/Cordova for Android build; Piskel/LibreSprite (for art if not fully AI-generated).
- **Integration Points:** Integrate Dhruv's code and Vinayak's assets into one build. Ensure Milind's Android export can run the game. Sync local storage (save state) code with game events (on exit).
- **Testing Checklist:** Verify player can move in all directions and collide correctly. Ensure stats bars (health/hunger/thirst) update. Test picking up sample resources (food/water). Confirm game runs fully offline (no crashes if disconnected). Deploy to phone and ensure the APK launches.
- **Handoff Notes:** Commit all code, art, and assets to repo. Merge tests for movement and resource pickup. Prepare documentation: "Prototype Build Notes" (includes controls, stats logic, assets list). This prototype will serve as the foundation for fleshing out survival systems in Phase 2.

Phase 2: Core Systems & Single-Player Features

- **Goal:** Build out the full single-player survival mechanics and environment generation. Implement procedural terrain, resource gathering, crafting, and inventory.
- **Duration:** ~5 weeks.
- **Deliverables:** Expanded game with multiple resources (water sources, food, wood), crafting menu, day/night cycle, weather effects, basic procedural map generation (e.g. random island/forest sections). Functional inventory and crafting UI. A first complete "mini-campaign" of quests (find water, build a fire, explore crash site).
- **Team Tasks:**
- **Dhruv (OpenCode/Nemtron/NVIDIA):** Use OpenCode to generate code for new survival stats (Stamina, body temperature) and resource tables. Prompt Nemtron to code the procedural map generation logic (using Perlin noise or tile algorithms). Implement a crafting system: define JSON

recipes (wood+stone → axe, etc.) and code the crafting UI. Use the NVIDIA model to auto-generate more complex sprites (trees, rocks, tools) from textual prompts. Write code to apply weather effects (rain, heat) to stats. Implement auto-save to device on checkpoints (using local file or WebStorage API).

- **Milind (Antigravity):** Integrate Bluetooth hardware as a simulated multiplayer layer stub for now: with Antigravity, draft code for a Bluetooth Manager class (scanning and connecting) as future work. Optimize core loops to run smoothly on target devices (profile code with Android Studio, reduce draw calls). Configure engine plugins for mobile sensors (e.g. accelerometer input to shake for certain actions). Continue building and testing on device; ensure offline save uses correct Android permissions (file write). Use Antigravity to generate and fix unit tests for inventory management and resource logic.
- **Vinayak (Antigravity):** Create and animate environment assets via Antigravity: generate sets of 8-bit tile sprites (ground types, water, trees). Use Antigravity to help write shader code or animations for day/night transitions (if engine supports shaders, otherwise use frame animations). Implement the quest UI and journal system in the engine, writing dialogs and quest descriptions (prompting AI for story flavor). Integrate Bosca Ceoil or SFXR for background music and sound effects (Antigravity can suggest melody themes or effects). Polish the UI: inventory screen, crafting menu, health bar design. Document the quest flow and game logic for the team.
- **Tools/Libraries/Languages:** Same engine; Node.js/npm (if Phaser) or Godot editor; IndexedDB or SQLite plugin for deep saves; Git/GitHub; Antigravity agents; Libresprite/Piskel for any manual touch-ups; Box2D physics (if needed for hunting or traps).
- **Integration Points:** Link the procedural world generator to the gameplay: objects placed in the map (water pools, trees). Integrate inventory/crafting code with UI implemented by Vinayak. Hook up persistent save after each quest objective. Prepare a test scene for each environment (tropical island, forest).
- **Testing Checklist:** Ensure resource nodes (trees, water) spawn and are gatherable. Test crafting: combine items to make tools/shelters. Validate health/hunger decrease/regeneration logic. Check day/night cycle and weather damage (heat or cold) affect stats correctly. Test save/load: progress is saved and restored accurately after quitting. All single-player features should work offline.
- **Handoff Notes:** Finalize detailed Game Design Document sections on survival mechanics and quests. Merge completed asset sets and code for resource/crafting systems. Provide integration instructions: how to generate new maps, adjust quests, and add new items. Phase 3 will add narrative and polish art, so assets should be organized and the codebase stable.

Phase 3: Content, UI & Narrative

- **Goal:** Flesh out game content: quests, story, artwork, animations, and audio. Create a more immersive world by adding multiple biomes and visual polish.
- **Duration:** ~3–4 weeks.
- **Deliverables:** A complete in-game episode or demo: handcrafted map sections for a hot island and dense forest, each with distinctive art and challenges. Full quest chain implemented via dialogue boxes or diary entries. Animated sprites (rain, fire, wildlife). Music and sound integrated. UI/UX finalized (menus, buttons, HUD styling).
- **Team Tasks:**
 - **Dhruv (OpenCode/Nemtron/NVIDIA):** Use Nemtron to script event sequences for quests and cutscenes (e.g. plane crash flashback, NPC discovery). Automate placement of narrative markers (journals, flares). Implement optimization for asset streaming (load scenes progressively). Refine graphics performance: use Nemtron or NVIDIA model to minify sprite atlases and compress textures. Integrate audio triggers (play thunder sound, music track) via code. Prepare a build that exports to WebGL (for iOS browser) and test the web version.

- **Milind (Antigravity):** Test how the game handles multiple scenes/levels (stress-test loading times, memory). With Antigravity, generate a partial code-base for an offline leaderboard or local save cross-check (for future use if Wi-Fi present). Focus on performance: profile on various phones, turn off unnecessary updates (e.g. don't animate offscreen objects). Use Antigravity to produce device test scripts (e.g. automated input sequences) for regression testing. Prepare build scripts for Android APK and iOS (Web export + wrapper) ensuring certificates are set.
- **Vinayak (Antigravity):** Lead the art pipeline: use Antigravity AI to expand the color palette and refine pixel art for each biome. Animate characters (e.g. player walking, animals). Implement particle effects (smoke, sparks) by having AI suggest shader or sprite-frame sequences. Integrate Bosca Ceoil loops and SFXR effects into the game engine. Polish UI elements: finalize fonts, layout for mobile screens, virtual joystick design. Conduct playtesting sessions, gather feedback on interface/usability.
- **Tools/Libraries/Languages:** Same engine; Inkscape or Krita (open-source) if needed for art; Bosca Ceoil, LMMS for music; Audacity (free) for sound editing; GitHub Actions for automated builds.
- **Integration Points:** Import all new assets and ensure they display correctly in scenes. Hook up dialogue/quest systems to the underlying code (trigger survival challenges as narrative objectives). Merge the Android and Web export pipelines: ensure Bluetooth code stub doesn't break the web build.
- **Testing Checklist:** Walk through every quest from start to end. Check that animations (e.g. rain, fire) play and loop smoothly. Ensure audio cues (ambient sounds, transitions) work. Test UI on target devices (buttons responsive, text legible). Verify platform exports run: Android APK installs and WebGL version runs in a mobile browser.
- **Handoff Notes:** Deliver all content assets (art, audio, story text) along with source files. Update design docs with final quest list and dialogues. Hand over performance reports and any identified bottlenecks. Prepare the game for the major multiplayer integration in Phase 4.

Phase 4: Multiplayer & Networking

- **Goal:** Implement local multiplayer over Bluetooth and ensure smooth host/client synchronization. Players should be able to start a game together (Host-Client), share a world state, and play cooperatively (or compete).
- **Duration:** ~4 weeks.
- **Deliverables:** A working multiplayer mode with Bluetooth connectivity: players can create or join a session, see each other's avatars in the world, and share resource data in real-time. All game state (positions, actions, inventory changes) remains in sync. Multiplayer menu/UI for connection.
- **Team Tasks:**
 - **Dhruv (OpenCode/Nemtron/NVIDIA):** Design and code the Bluetooth P2P network layer. Use Nemtron to outline the **Host-Client sync model**: one device is authoritative (Host), others send input to it. Implement message protocols (position updates, resource pickups) over BLE. Apply **client-side prediction** and **reconciliation** (using techniques from multiplayer networking [15†L35-L43] [15†L108-L117]) so that movement feels smooth despite latency. Use OpenCode to generate test harnesses that simulate packet loss or delay. Integrate the new networking code with existing game logic (e.g. linking input events to network messages).
 - **Milind (Antigravity):** Set up physical testing: pair multiple phones via Bluetooth. Use Antigravity AI to write scripts that automate the connection process (host discovery, pairing). Debug connection stability (reconnect if link drops). Use the hardware skillset to ensure compatibility: manage Android permissions (BLUETOOTH_ADMIN, location); on iOS, use Multipeer as fallback if needed. Optimize Bluetooth throughput: compress game state updates, minimize message size.

Monitor battery usage during Bluetooth play and apply power-saving measures (e.g. cap network tick rate).

- **Vinayak (Antigravity):** Build the multiplayer UI: “Host Game” and “Join Game” menus, list of nearby devices, connection status indicators. Use Antigravity to generate icons and instructions for joining by Bluetooth. Implement in-game indicators (e.g. player names above avatars, chat or ping UI if feasible). Coordinate on gameplay balance: adjust resource respawn or add PvP mode rules if competitive mode is desired. Test user experience of “connection lost” and “new player joined” events, and make the UI intuitive. Document the multiplayer flow and controls.
- **Tools/Libraries/Languages:** Cordova-plugin-ble (Apache-2.0) for BLE connectivity; or native BLE API for Godot; MultipeerConnectivity for iOS if using native; Phaser’s network event loop or Godot’s High-Level Multiplayer API (if using Godot) as base. Use OpenCode’s CLI to assist in writing networking scripts. Use Node.js for any local testing servers if needed.
- **Integration Points:** Integrate network code into the main game loop: inputs send to host, host applies authoritative game updates to all clients. Ensure Bluetooth library is initialized at game start and persists throughout scenes. Tie multiplayer state to the save system (e.g. for co-op progression).
- **Testing Checklist:** Pair two or more devices and start a session. Move on one device; verify the other sees it move smoothly (with prediction compensating lag) 【15†L35-L43】. Test resource sharing: if Host picks up an item, clients should see it removed. Verify that simultaneous actions (both clients moving, crafting) sync correctly. Test scenario: host drops out (what happens?). Check battery draw: game should pause or throttle if app in background. Validate that multiplayer still works offline (no network needed).
- **Handoff Notes:** Finalize and comment networking code. Provide a “Bluetooth Multiplayer Guide” summarizing how to add new synced variables or handle new actions. Ensure code paths fallback gracefully if Bluetooth is unavailable (so single-player still works). Prepare builds that include Bluetooth plugin settings. The game is now feature-complete; Phase 5 will focus on polish and deployment.

Phase 5: Optimization, Polishing & Release

- **Goal:** Ensure the game is stable, performant, and ready for release. Perform final optimizations, platform-specific tweaks, and compliance (store submission assets).
- **Duration:** ~2–3 weeks.
- **Deliverables:** Fully tested final builds for Android, iOS (WebGL HTML5 export or native), and WebGL. Store-ready package with icons, description, and a post-launch update plan.
- **Team Tasks:**
 - **Dhruv (OpenCode/Nemtron/NVIDIA):** Perform code profiling and optimization: use Nemtron to refactor slow functions, reduce unnecessary object allocations. Use NVIDIA AI to generate optimized code snippets (e.g. math libraries for physics). Integrate battery-saving modes: throttle frame rate to 30 FPS on older devices. Finalize the save/load system for version 1.0 (e.g. cloud backup if Wi-Fi exists, but optional 【20†L124-L133】). Export builds: for Android (generate signed APK), for iOS (ensure WebGL build runs on Safari’s offline mode), for Web (compress assets for WebGL). Write scripts (OpenCode) to automate building for each platform.
 - **Milind (Antigravity):** Conduct extensive device testing across models. Use Antigravity to run automated stress tests on low-end hardware (simulate memory limits). Optimize power: disable sensors/audio when idle, remove debug logs. Ensure Bluetooth session does not drain battery excessively (see Android battery profiling). Work on packaging: prepare final APK and test on Google Play’s validation, ensure proper handling of permissions. Review analytics and crash logs (if any) to fix issues.
 - **Vinayak (Antigravity):** Polish the UX: fix any remaining UI bugs (text cutoff, button placement). Add polish animations (e.g. inventory screens sliding) and polish art (anti-alias where needed for

high-res screens). Create marketing materials: screenshots, trailers (recorded gameplay) guided by Antigravity for captions. Set up social media and App Store/Play Store listings with AI-generated descriptions. Collect playtester feedback from final builds and implement minor balance tweaks.

- **Tools/Libraries/Languages:** Android Studio (free), Xcode (for iOS WebGL build via PhoneGap or similar), gzip/texture compression tools (e.g. ETC2, WebP); Google Play Console; TestFlight (iOS); analytics (Firebase or free alternative) integration. Version control (GitHub) for final release tags.
- **Integration Points:** Coordinate with any external testers or final QA. Ensure the core offline/multiplayer works identically in release mode. Merge final localization or optional online features (leaderboards via Wi-Fi) if decided.
- **Testing Checklist:** Full regression: play the entire game on all modes (solo coop, multi coop, multi vs). Test installation/uninstallation, app lifecycle (pause/resume). Battery check: the game should not heat devices unduly after 1h play. Bluetooth sessions tested to max players (2–4). Validate saved games load correctly after app kill/restart. Perform compliance checks (age ratings, privacy).
- **Handoff Notes:** Tag the final release in version control and document all build steps. Provide a “Release Dossier” with credentials for stores, asset guidelines, and a post-launch support plan. Highlight any known issues for the next update. Prepare a brief post-mortem outlining future expansion possibilities (new biomes, quests, online multiplayer) as part of project debrief.

Sources: This plan draws on game engine documentation and best practices. Phaser, Godot, and GDevelop are all free/MIT licensed [2†L7-L10] [7†L330-L338] . Bluetooth connectivity in browsers is limited [13†L25-L28] , so native P2P layers (BLE) will be used. Techniques like client-side prediction and server reconciliation (authoritative host model) are standard in peer-to-peer games [15†L35-L43] [15†L108-L117] . Offline-first design mandates local data persistence and immediate UI response [20†L124-L133] . All recommended tools (Phaser, Godot, GDevelop, Piskel, Bosca Ceoil, SFXR, etc.) are free/open-source, aligning with project constraints. The phased timeline ensures a working single-player foundation before layering on content and multiplayer, with continuous integration and testing at each step. Tables and mermaid chart above visualize engine choices and schedule.
